PROTOTYPE · SAMPLE DATA · TEST PAYMENTS

Safar Saathi

An offline-first travel companion for first-time international travellers from India. Project guide: technology, architecture, data and process flows.

Safar Saathi helps a traveller **after** they have booked: it keeps their plan, map, help guides and SOS working with no internet, an AI agent answers questions and prepares changes, and a human support agent is alerted when something goes wrong. Its main feature turns any set of existing travel documents (PDF, Word, Excel, images) into a complete offline trip.

> **Live site:** safar-saathi-five.vercel.app (Vercel) · **Source:** github.com/nupurjethi/Safar-Saathi · **Database:** Neon Postgres · **AI:** Gemini through OpenRouter

Contents

1. What the product does
2. Technology stack at a glance
3. System architecture
4. Front end
5. Back end and API
6. Databases and data model
7. The offline system (how offline storing works)
8. The offline map and routes
9. The Add Travel Plan agent (documents to offline trip)
10. The Trip agent, tools and the Confirm gate
11. Offline assistant and help guides (rule engine)
12. SOS, SMS and the agent desk
13. Flights, hotels and booking-platform handoff
14. Accounts, sessions and security
15. Scroll story and visual design
16. Testing, deployment and operations
17. Known limits and honest boundaries
18. End-to-end user journeys

1. What the product does

The product has four jobs, each of which works with or without internet:

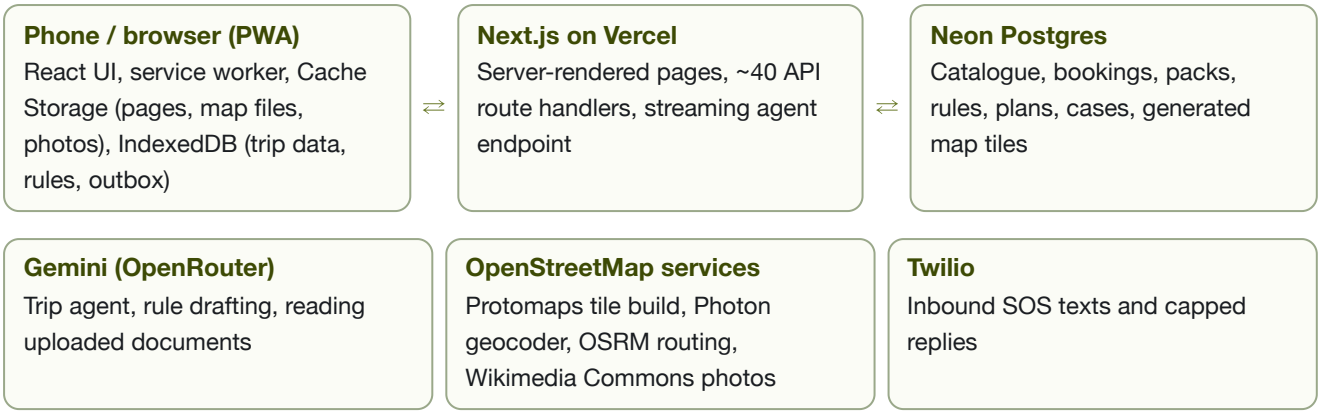
- **Plan:** a catalogue of 113 sample itineraries across 26 cities, plus the ability to upload your *own* bookings and have the app build the trip from them.
- **Travel offline:** every trip is saved on the phone as an offline bundle: day plan with real photos, a real street map with a GPS dot, saved road routes, six validated help guides, emergency contacts, an offline assistant and an SOS that queues until signal returns.
- **Get help:** an online AI Trip agent answers from the booking and prepares changes that only the traveller can confirm; SOS taps and SMS texts open cases on a human agent's desk.
- **Change plans honestly:** flights and hotels are changed on the traveller's own booking platform (MakeMyTrip, Goibibo, Cleartrip, ixigo). Safar Saathi compares sample prices, sends the traveller there to pay, and records the result after a Confirm tap.

> **Honesty rules built into the product:** all catalogue trips, flights, hotels and prices are sample data from our own mock API; no travel site is scraped; the app never asks for or stores a booking-platform password; payments are test payments; offline guidance is predefined, never live information.

2. Technology stack at a glance

Layer	Technology (version)	Why it is used
Framework	Next.js 16 (App Router, Turbopack), React 19, TypeScript	Server components for fast first load, route handlers for the API, one codebase for web and PWA.
Styling / UI	Tailwind CSS 4 with CSS-variable design tokens, Outfit font (next/font), lucide icons	Neon-green on near-black theme, light and dark modes, mobile-first.
3D / animation	Three.js, GSAP + ScrollTrigger, Lenis smooth scroll	The scroll story: a procedural airliner, a waving cloth banner and cloth trip cards.
Offline storage	Service worker (hand-written), Cache Storage, IndexedDB through Dexie 4	App pages and map files in Cache Storage; trip data, rules and the outbox in IndexedDB.
Maps	MapLibre GL 6, PMTiles 4, Protomaps basemap styles (OpenStreetMap data), local fonts and sprites	A real, Google-Maps-like street map that works offline with GPS.
Routing	OSRM public servers (OpenStreetMap routing) at build time; OpenRouteService optional	Real road and footpath routes with turn-by-turn steps, pre-computed so phones need no routing service.
Database	Neon serverless Postgres in production; PGlite (Postgres in WebAssembly) locally and in tests	Same SQL and schema everywhere; no database needed to run locally.
ORM / migrations	Drizzle ORM 0.45 and drizzle-kit (SQL migrations 0000 to 0005)	Typed queries and versioned schema.
AI	Vercel AI SDK 7; Gemini (gemini-3.5-flash) through OpenRouter; Anthropic/OpenAI adapters	Tool-calling Trip agent, rule drafting and document understanding (text and images).
Document reading	unpdf (PDF), mammoth (Word), exceljs (Excel), plain parsing for CSV, HTML, text; images go straight to the model	Any common travel document can be an input.
Validation	Zod 4	Every API body, every model answer and every rule set is schema-checked.
Auth	jose (signed JWT cookie), scrypt password hashing (node:crypto)	Sessions without an extra auth service; no password stored in plain text.
SMS	Twilio REST (API key), polling of inbound texts	SOS by text when there is cellular signal but no data.
Testing	Vitest (unit, ~256 tests), Playwright (end-to-end, 14 tests incl. airplane mode)	Every claim in the pitch maps to a test.
Hosting	Vercel (Pro) for the app; Neon for data; GitHub for source	Serverless deployment; map files served from the same site.

3. System architecture



Key design idea: the phone never depends on a server to *use* a trip. The server builds an **offline pack** (a single JSON document plus a list of files), the phone downloads and verifies it, and from then on every screen reads from the phone's own storage. The server is needed only to download or update the pack, to talk to the AI agent, to confirm changes and to upload SOS cases.

Repository layout (main folders)

`app/`	Next.js routes: pages (`/` , `/explore` , `/itinerary/[slug]` , `/trips/[ref]/…` , `/plans/new` , `/agent` , `/admin`) and `app/api/**` route handlers.
`components/`	UI: chat (agent, offline assistant, review/compare/hotel/platform cards), trip screens, map, story, plans, agent desk.
`lib/`	All logic: `db` , `packs` , `offline` , `rules` , `agent` , `gate` , `plans` , `providers` , `map` , `sms` , `seed` , `server` , `story` , `llm` .
`public/`	Service worker `sw.js` , map tiles/fonts/sprites, landmark photos, icons.
`drizzle/`	SQL migrations. `scripts/` : seed, tile build, route cache, photo fetch.
`tests/`	Unit and end-to-end tests.

4. Front end

4.1 Rendering model

Pages use the Next.js App Router. Content that does not depend on the traveller (catalogue, itinerary pages) is rendered on the server. Trip screens are **client-rendered shells with no personal data in the HTML**: the HTML is identical for everyone, and the data is read from IndexedDB. This is what allows the service worker to cache the page HTML safely and serve it with no network.

4.2 Main screens

- **Landing (`/`):** visitors see the "How it works" scroll story with Travel user login and Agent login. A signed-in traveller sees Home (the trip catalogue); an agent is redirected to the agent desk.
- **Explore / itinerary:** search by destination, month, budget, theme and tier; itinerary pages show a day-by-day plan, real landmark photos with credits, a real street-map preview with city tabs, a price breakdown and the offline-size estimate.
- **My trips:** trips with photos, "Offline ready" badges, **Add travel plan** and a delete button (with confirmation) on every trip.
- **Trip screens (`/trips/[ref]/…`):** Home (hero photos, next plan, pickup card with code, hotel card, local time, weather online), Chat (online Trip agent), Offline (separate offline assistant), Guide (six problem guides), Map, SOS, Log (action log) and Voucher (printable).
- **Add travel plan (`/plans/new`):** multi-file upload with per-file status and live processing steps.
- **Agent desk (`/agent`):** live alert list for support staff.

4.3 State and data access

There is no global state library. A `TripProvider` loads the saved pack from IndexedDB and runs a quiet background sync whenever the device is online. A network-state hook distinguishes "Online", "Offline pack" (no data, trip saved) and "Signal only". The chat keeps its own local message state; the offline assistant keeps its history in `localStorage` per trip.

4.4 Mobile and accessibility

Tap targets are at least 44 px, inputs are 16 px (prevents iOS zoom), fixed bars respect safe-area insets, long names wrap, and horizontal chip rows scroll rather than overflow. Reduced-motion and no-WebGL fallbacks exist for the story.

5. Back end and API

The back end is Next.js route handlers running as serverless functions on Vercel (Node runtime). Every handler is wrapped in a common `handle()` helper that turns typed errors (`GateError` with a code and HTTP status) into JSON responses, and bodies are validated with Zod.

Area	Endpoints (examples)	Purpose
Auth	`/api/auth/demo | register | login | otp | logout`, `/api/me`	Demo login, email + password, email code, session.
Catalogue	`/api/trips/select`, `/api/trips/[ref]/pay`, `DELETE /api/trips/[ref]`	Select (creates a draft and builds the pack), test payment, delete.
Offline packs	`/api/packs/[ref]/manifest`, `/api/packs/[bookingId]/pack`	The file list with hashes and the pack JSON.
Travel plans	`/api/plans`, `/api/plans/[id]/files`, `/api/plans/[id]/process`, `/api/tiles/[key]`	Upload files one by one, stream processing progress, serve generated map files.
Agent	`/api/agent` (NDJSON stream)	Tool-calling Trip agent with live step chips.
Change gate	`/api/confirmations`, `/api/mock/v1/changes/apply`, `/api/trips/[ref]/external-change`, `/external-hotel-change`	Quote, Confirm token, apply, record platform bookings.
Mock travel API	`/api/mock/v1/flights/search`, `/stays/availability`, `/quotes`, `/bookings/[ref]`	Our own sample partner API.
Help	`/api/cases`, `/api/sos`, `/api/sms/inbound`, `/api/agent/alerts`, `/api/cron/sms-poll`	SOS upload, SMS, agent-desk feed.
Platforms	`/api/providers/connect`	Remembers which booking platform the traveller says they are logged in to.

Cross-cutting server pieces

- **Rate limiting:** a sliding-window limiter per user or IP (in memory per serverless instance; adequate for a prototype, swap for Redis in production).
- **Security headers:** a strict Content-Security-Policy (own origin only, plus blob storage and workers), no framing, nosniff, referrer policy, geolocation limited to self.
- **Single LLM module:** every model call lives in `lib/llm/index.ts` (model choice, logging, retries, test hook). A test fails the build if any other file imports an AI SDK.

6. Databases and data model

6.1 Which databases

- **Neon (serverless Postgres):** production data. The app connects with the Neon serverless driver over WebSocket pooling, which supports transactions.
- **PGlite:** a real Postgres compiled to WebAssembly, used when `DATABASE_URL` is empty. Local development, the end-to-end tests and the unit tests (in-memory) all run on it with the same schema and queries. This is why the project runs with no keys.
- **IndexedDB (on the phone, through Dexie):** the traveller's offline copy: trips, pack versions, rule sets, per-file records, and the outbox of queued SOS cases and chat messages.
- **Cache Storage (on the phone):** pages, scripts, map files, photos.

6.2 Main tables (Drizzle schema, ~35 tables)

Group	Tables	Holds
Catalogue (mock API)	destinations, pois, hotels, flights, room_inventory, transfers, itineraries, itinerary_days, mock_hotels, partner_markups	26 cities, real landmarks with coordinates and licensed photo credits, fictional hotels, ~100,000 generated flights, room inventory, 113 itineraries, per-platform price rules.
People	users (role traveller | agent | admin, password hash), login_codes, travellers	Accounts and the people on a trip.
Trips	bookings (source catalogue | upload), booking_items, payments, trip_drafts, trip_versions	A trip, its flights, hotels, transfers and activities as items, payments (including recorded external ones).
Uploaded plans	travel_plans, plan_files, map_tiles	The extracted plan, files' text (deleted after processing), generated PMTiles bytes.
Offline	offline_packs (pack JSON + manifest + version), rule_sets, rule_validations	Versioned bundles and the validated help guides with their check results.
Agent + gate	agent_threads, agent_messages, tool_calls, change_quotes, price_breakups, confirmations	Chat history, an audit log of every tool call, quotes with hashes, single-use confirmation tokens (hash only).
Safety	cases, sms_messages, sos_cases_local_sync, events	Help cases with SLA, SMS in/out, offline-SOS de-duplication, analytics events.
Integrations	provider_offers, provider_connections, llm logging via events	Provider offers, "logged in to" notes.

6.3 Seeding and determinism

The catalogue is generated by a seeded random generator (`lib/seed`) so every environment gets identical data. Bangalore draws from its own random stream so adding it did not change any other city. `npm run db:reset` drops the schema, migrates and reseeds (about two minutes on Neon). Demo accounts: a demo traveller, a demo support agent and an admin.

7. The offline system (how offline storing works)

> **Principle:** select first, pay later. The moment a traveller selects a trip, the whole offline bundle is built and saved on the phone, before any payment.

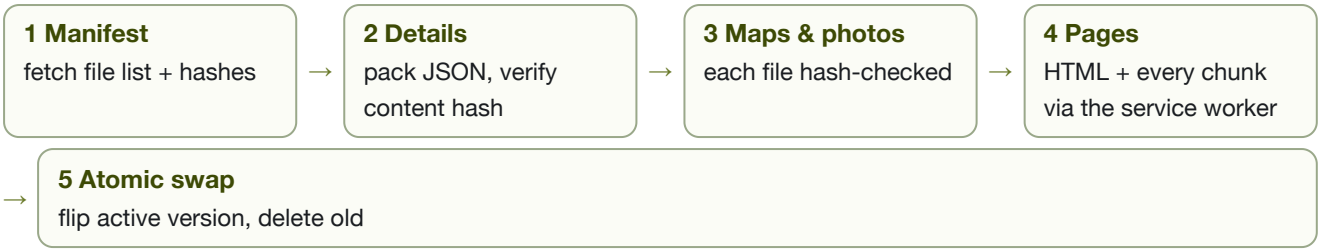
7.1 What is stored and where

Item	Stored in	Detail
Trip data (the "pack")	IndexedDB `tripPacks`	One JSON document: booking, days, places (with coordinates and photo credits), stays, flights, contacts, saved routes with steps, map references.
Help guides	IndexedDB `rulesets`	Validated decision trees for six scenarios, filled with the trip's own facts.
Map files (PMTiles)	Cache Storage `ss-trip-{ref}-v{n}`	Overview + street-level core files per city (5 to 28 MB each).
Landmark photos	Same per-trip cache	Openly licensed photos with credit text.
Fonts, icons, map worker	Cache Storage `ss-shell-v2`	Shared by every trip (about 3.4 MB).
App pages and JavaScript	`ss-shell-v2` and `ss-static-v1`	Trip pages (home, chat, offline, guide, map, SOS, log, voucher), My trips, Explore, Add plan and every JS/CSS chunk they need.
Queued SOS and chat	IndexedDB `outbox`	Sent automatically when the device is back online; idempotent by local id.
Bookkeeping	IndexedDB `trips`, `fileRecs`	Active version, per-file hash and size so interrupted downloads resume.

7.2 Building the pack (server)

1. Resolve the booking items (flights, hotels, transfers, activities) and the destination data.
2. Build places, day-by-day items with local times in each city's time zone, stays, flights and contacts.
3. Compute saved routes (airport to hotel both ways, hotel to each landmark) from the pre-built real-route cache.
4. List the map tiles, fonts, photos and pages; compute each file's size and SHA-256.
5. Draft the six help guides with the LLM (or fall back to templates), validate them (section 11), freeze them.
6. Hash the whole content; if unchanged from the last version, no new version is created; otherwise store version n+1.

7.3 Downloading the pack (phone): the sync algorithm



- **Order:** trip data and guides first, so the guide works before the maps finish.
- **Integrity:** a file is written to the cache only after its SHA-256 matches; a bad file throws and writes nothing.
- **Resume and reuse:** every verified file is recorded immediately; an interrupted download resumes; files unchanged since the previous version are copied from the old cache instead of downloaded.
- **Atomic swap:** all new files go under version n+1; the active version flips in one IndexedDB transaction only after everything verified. If anything fails the previous version stays active and usable.
- **Budget:** 80 MB per trip by default; over budget offers "download maps over Wi-Fi".
- **Self-healing:** on every online arrival the app re-checks that every saved page, map file, photo and font still exists (iOS can delete storage) and re-downloads only what is missing, even when the pack itself is unchanged.
- **Update downloads bypass the saved copy:** downloads carry `?ss-refresh=1` so the service worker does not answer from the old cached file (otherwise an updated map could never replace an old one).

7.4 The service worker (`public/sw.js`)

- **Pages:** network-first with a 4-second timeout, falling back to the saved copy of the page; unknown pages fall back to a friendly `/offline` page listing the saved trips.
- **Static assets (`/_next/static/*`):** cache-first (hashed, immutable). When saving a page, the worker scans its HTML and then every chunk's text for `static/chunks/…` references, so lazily loaded code (like the map) is saved too.
- **Map files (`.pmtiles`, `/map/`, `/places/`):** served from the caches with HTTP **Range** support. PMTiles reads only small ranges; the worker slices the cached file to answer.
- **API calls:** never cached (authenticated data). Next.js client-side route fetches are not intercepted; offline the app uses a full page load instead (`SafeLink`), which the worker serves from the saved pages.
- **Versioned caches:** per-trip namespaces `ss-trip-{ref}-v{n}` so deleting a trip removes only its own files.

7.5 Network states and honesty

The header always shows the network state. SMS needs cellular signal and does not work in airplane mode; GPS works with no data; offline limits (no live traffic, no address search, navigation pauses in the background) are said plainly in the UI.

7.6 Device-specific caveats

On iOS the installed Home Screen app has storage separate from Safari: the trip must be downloaded inside the installed app after signing in there. iOS may evict stored data under pressure; the self-healing check repairs this the next time the app is opened online.

8. The offline map and routes

8.1 Tiles

- Source: the daily **Protomaps** planet build (OpenStreetMap vector tiles). The `pmtiles` CLI extracts a bounding box with HTTP range requests, so no bulk download from tile.openstreetmap.org (forbidden by its usage policy).
- Two files per city: an **overview** (whole city and airport, zoom 0 to 12; 14 for Bangalore) and a **core** file (hotels and landmarks, street level to zoom 15). All 26 cities total about 250 MB on disk; a two-city trip stores about 30 to 45 MB.
- The core bounding box is stored in the tile manifest, so the server never opens the large files at run time.
- **Generated tiles:** for a city in an uploaded plan that we do not cover, the server builds a small PMTiles v3 file in-process (reads only the needed tiles from the public build, re-packs them, serves it from `/api/tiles/<key>.pmtiles`, stores it in Neon).

8.2 Style and rendering

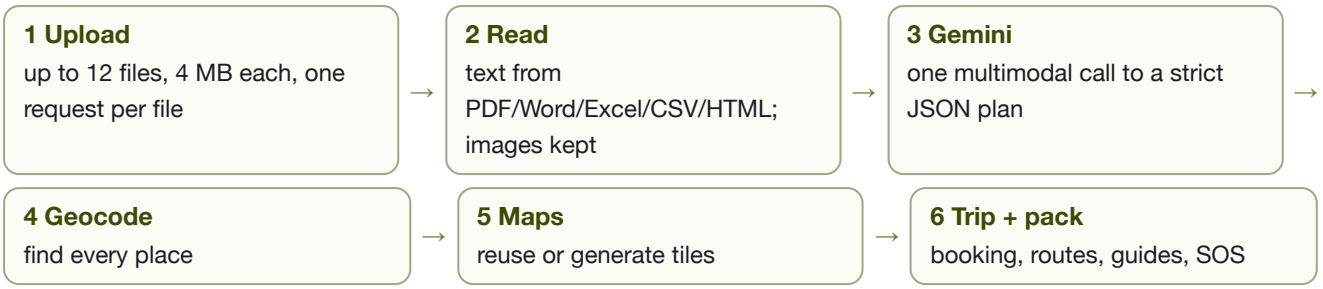
MapLibre GL draws the style produced by `lib/map/style.ts` from Protomaps layers (roads with casings, buildings, parks, water, rail, one-way arrows, street names, POI icons). The style uses only local URLs: tiles from the cached PMTiles, glyphs from `/map/fonts` and icons from `/map/sprites`, so nothing calls the network offline. Core layers draw on top from zoom 13 and hide the coarser overview underneath. Light and dark styles are tuned for contrast.

8.3 Pins, routes and GPS

- **Pins:** teardrop markers for hotel (H), airport and numbered landmarks; the traveller's position is a neon-green dot with dark rings, a pulse and an accuracy circle.
- **Routes:** real road and footpath geometry plus turn-by-turn steps, pre-computed from OSRM into `lib/seed/route-cache.json` (954 routes) and saved in the pack. An off-route detector warns when the traveller is more than 40 m away for 10 seconds.
- **GPS:** the browser Geolocation API works with no data. "Start navigating" keeps the screen awake while the page is visible.

9. The Add Travel Plan agent (documents to offline trip)

This is the main agent: a traveller uploads whatever they already have and the backend produces the complete offline bundle from it.



Step 1 and 2: uploading and reading

The browser creates a plan (`POST /api/plans`), then posts each file separately (`/api/plans/[id]/files`) so every request stays within the serverless body limit and each file shows its own status. `lib/plans/parse.ts` extracts text with `unpdf`, `mammoth`, `exceljs` (every sheet as rows) or direct decoding; photos and screenshots become data URLs the model reads directly. Scanned PDFs with no text are reported ("upload a photo instead"); unsupported types are skipped with a reason. Only text and images are stored, and only until processing ends.

Step 3: Gemini extraction

`lib/plans/extract.ts` sends all texts and images in one call with strict instructions: use only facts in the documents, never guess phone numbers, booking codes or times; local date-times; IATA codes; time zones; hotels, activities, transfers, contacts. The JSON is normalised (upper-case codes, empty strings removed) and validated with Zod (`lib/plans/schema.ts`); on failure it retries once with the validation errors. Plain-English **warnings** (no hotel phone, no flights found) are produced and shown to the user.

Step 4: finding places on the map

`lib/plans/geo.ts` looks first in our own data (seeded airports, cities, landmarks), then asks the public **Photon** OpenStreetMap geocoder (one request per second, results kept only if near the city). A place not found gets no pin and a warning; the app never guesses coordinates.

Step 5: offline street map

Cities we already cover reuse our pre-built tiles; any other city gets a map file generated at that moment (section 8.1). Goa, Kyoto and Osaka were built this way: 3 to 5 MB each in 15 to 40 seconds.

Step 6: trip and bundle

A booking with `source = "upload"` is created, with flight and hotel items. `lib/plans/pack.ts` builds the same offline pack as a catalogue trip: day plan from the documents' dates and items, pins, real routes (live OSRM for new places), stays, drivers and pickup codes from the transfers, contacts, emergency numbers, six validated guides and the map references. The documents' text and images are then deleted. A plan with no flights (for example a rail itinerary) still builds, with "not in your documents" placeholders.

> **Typical timing (live test):** reading with Gemini 13 to 35 s, finding places 10 s, generating one city map 15 to 40 s, building the bundle a few seconds; about 1.5 minutes in total, inside the 5-minute function limit on Vercel Pro.

10. The Trip agent, tools and the Confirm gate

10.1 The agent

The online Trip agent is a tool-calling model (Gemini through OpenRouter, via the AI SDK). It answers only from tool results. It streams NDJSON events to the browser: thread id, step chips (which tool is running), text deltas, cards (review, compare, hotels, platforms, case, route) and a final label. Without an API key a deterministic "demo agent" runs the same tools through the same gate and is labelled in the UI.

Tool	What it does
get_itinerary	Reads the trip: next plans, pickup, hotel, local time, flights with ids.
get_route	Returns a saved route and steps; shows an "Open on map" card.
compare_flights / compare_hotels	Compares prices (official APIs when keys exist, otherwise our mock partner API in Neon) with a per-platform price for each option.
connect_platform	Shows the four-platform picker and the form to record a booking.
quote_change / apply_change	Price a change and apply it; apply needs a valid Confirm token (see below).
search_web	Travel information with cited sources (Tavily/Brave when configured); never a booking price.
escalate_human	Opens a case for a person (severity from a rule-based rubric); the agent can never close a case.
send_sms	Short SMS to the traveller (max 160 characters, capped).

A test asserts there is **no tool that can mint a confirmation token**, and that the model's allow-list excludes server-only jobs.

10.2 The Confirm gate (nothing changes without the traveller)

1. **Quote:** the server computes the change and the price breakup; a hash of the quote (kind, parameters, before/after, price lines) is stored.
2. **Review card:** the traveller sees before/after, the price breakup, the source ("Sample data" or "Entered by you") and Yes / No.
3. **Confirm tap:** an authenticated request mints a single-use token; only its hash is stored; it expires quickly.
4. **Apply:** requires the token; it is claimed atomically; the quote hash is re-checked; the price is re-computed from live data. If it moved, the old quote is voided and a new review card is shown (HTTP 409).
5. **Mutate and refresh:** items are replaced in one transaction, payments recorded, then the offline pack is rebuilt (only the affected guide scenarios are redrafted) and the phone re-syncs.
6. **Limits:** changes above a price limit are blocked and escalated to a person; changes within 3 hours of departure are refused.

Four change kinds go through the same gate: `flight_change` and `hotel_extend` (sample inventory, API only), and `flight_external` / `hotel_external` (a booking the traveller already made on a platform, recorded after Confirm; no price limit because Safar Saathi charges nothing).

11. Offline assistant and help guides (rule engine)

11.1 The six guides

Flight delayed, missed connection, hotel not found, transfer no-show, lost passport and medical help. Each is a short decision tree with steps filled from the trip's own data (hotel address in the local script for a driver, emergency numbers, consulate, pickup code, flight numbers).

11.2 Drafted once, validated, then frozen

1. When facts change, only the affected scenarios are redrafted by the LLM (one call, JSON only).
2. **Seven validators** run: schema (unknown fields fail), references (every `{{template}}` resolves in every city of the trip), coverage, no invented facts, length and tone, local script present, safety.
3. One retry with the error list; if it still fails, a **template rule set** labelled "Standard guide, not personalised yet" is used.
4. The validated set is stored, frozen and shipped to the phone. A tiny interpreter (under 5 KB) walks it offline. **No model and no network are used offline.**

11.3 The offline assistant (separate chat)

`/trips/[ref]/offline` is its own chat with its own history on the phone, not mixed with the online agent. It is a pure rule-based answerer (`lib/offline/assistant`): about 31 intents with typo tolerance and a few Hinglish phrases, answers read only from the saved pack (hotel, flights, pickup, today's plan, emergency numbers, per-country basics, the six guides), always citing the source item. When unsure it says "I can't answer that offline", and for changes it says they need a connection and the Confirm tap. A question can be queued for the online agent.

12. SOS, SMS and the agent desk

12.1 SOS from the app

The SOS screen shows the address card in the local script, emergency numbers and the SOS button. Tapping it while offline saves the case in the IndexedDB outbox with the last known GPS position; when connectivity returns it uploads (`/api/cases`), de-duplicated by the phone's local id, and the traveller sees the server case reference.

12.2 SOS by text message

1. The traveller texts the helpline number (Twilio) e.g. "SOS lost passport SS-48213". SMS needs cellular signal, not data.
2. A local server has no public webhook URL, so the server **reads the text from Twilio** by polling Twilio's message list (a free read) whenever an agent desk is open, or from a cron route.
3. The same handler as the webhook runs: severity rubric, booking matched by sender number or by the booking reference in the text; unknown senders still raise an alert.
4. A case is opened (severity 1 gets a 60-second SLA) and a short reply (160 characters) is sent.
5. **Message budget:** every real outbound SMS is counted in the database and refused after a cap (default 5), because trial accounts have few messages.

12.3 The agent desk (`/agent`)

Support staff sign in as agent (demo account for now). The desk refreshes every 5 seconds from `/api/agent/alerts` and shows each alert with severity, channel (SOS from app, SMS, help request from chat), the traveller and masked phone, the trip, the last GPS location (OpenStreetMap link) and an SLA countdown. New alerts raise a red banner, an optional alarm sound and a browser notification. "Take over" and "Mark resolved" are human-only actions.

13. Flights, hotels and booking-platform handoff

13.1 What is and is not possible

> MakeMyTrip, Goibibo, Cleartrip and ixigo give other apps **no sign-in, no price feed and no booking API** (their APIs are for contracted business partners). So Safar Saathi does not log in to them, read their prices or take payment, never asks for passwords, and never scrapes them. Real in-app booking would need a partner agreement and an official adapter.

13.2 The flow implemented

1. At the top of the chat the traveller picks **"I'm logged in to"** one platform (a self-declared note stored in `provider_connections`).
2. For "move my flight" or "extend my stay" the agent runs a comparison. Prices come from the **mock partner API stored in Neon**: seeded flights and room inventory plus flights and hotels generated deterministically for new routes or cities and stored once; `partner_markups` turns one base price into

a price per platform (markup, discount and fee). Every price is labelled as sample data. If an official API key (Amadeus or Duffel) is configured, live fares are used instead.

3. The comparison card shows ranked options; the primary button is **"Pay on <your platform>"**, which opens that site (flights pre-filled; hotels open the hotel page with the city and dates shown to copy). The traveller logs in and pays there.
4. Back in the app they press "I have paid: record my booking", check the pre-filled form (flight number, times, PNR, amount; or new check-out / chosen hotel), and review the Confirm card. Only their Yes updates the trip, totals and offline guide.

14. Accounts, sessions and security

- **Login options:** demo traveller, email + password (create account / sign in), email code (collapsed), demo agent and admin.
- **Passwords:** scrypt with a random 16-byte salt (`node:crypto`), only the hash is stored and is never returned by any API; weak or common passwords are refused; one generic error for every sign-in failure and a dummy hash run so timing does not reveal which accounts exist; rate limits per email and per IP.
- **Sessions:** a signed JWT (HS256, `jose`) in an httpOnly, SameSite cookie, 30 days.
- **Authorization:** every trip API checks the booking belongs to the session; staff routes require agent or admin; traveller-only routes (plans, delete) require the traveller role.
- **Data minimisation:** uploaded documents are deleted after processing; phone numbers are masked in staff views; SMS bodies are purged after 30 days by a cron route.
- **Honest labelling:** sample data, test payments and stubbed integrations are labelled in the UI and the README; demo mode switches exist and must be turned off for a real launch.

15. Scroll story and visual design

The landing page is a scroll-driven story built with **Three.js** (one shared stage), **GSAP ScrollTrigger** (pinned sections and scrubbed timelines) and **Lenis** (smooth scrolling). The airliner is built entirely in code: a lathe-turned fuselage with a livery texture, lofted swept wings and winglets, tailplane and fin, engines, gear and lights. The hero banner is a cloth-simulated pennant (40x14 vertex mesh with travelling waves, shaded folds, reaction to scroll speed); the trip decks wrap over the cabin roof, slide off along a curve while rippling, and settle flat and static in front. The flight path never descends. Reduced motion and no-WebGL users get a plain layout.

The theme is neon green (#d8fd34) on near-black (#050505) with Outfit, pill buttons and a pill navigation bar, driven by CSS variables so light and dark themes share one set of tokens.

16. Testing, deployment and operations

16.1 Tests

- **Unit (Vitest, ~256 tests):** seed determinism and counts, the gate (no token, reused, expired, foreign, tampered, price move, over limit), rule engine and validators, severity rubric, off-route detector, offline sync (with a fake IndexedDB and Cache API), agent tools and demo agent, Twilio budget and polling (mocked), the plan parser, schema and full plan-to-pack agent, partner pricing, map style (no remote URLs), cloth maths, accounts and password hashing, trip deletion, "no scraping" checks, and that all 113 itineraries select, pay and build a complete bundle.
- **End to end (Playwright, 14 tests, Pixel 7 profile):** explore, select, offline guide ready, airplane-mode guide/map/SOS with no failed requests, the neon GPS dot placement, platform compare and record, the Confirm gate, SMS to an agent-desk alert, the offline chat, sign-up and sign-in, plan upload, offline page switching and self-healing, deleting a trip. E2E runs on a production build with an isolated local database, and its configuration forces empty database and Twilio settings so a test can never touch production data or send a real SMS.

16.2 Deployment

- Source on GitHub; deployed with the Vercel CLI to a Pro team (the app has about 60 server routes, above the free plan's 12-function limit).
- Environment variables (database, AI key, Twilio, auth and cron secrets, demo-mode flags, price limit) are set in the Vercel project. Next.js file tracing is configured so runtime-read files (route cache, tile manifest, photos, fonts) ship with the functions.
- The large map files (~250 MB) are not in git; they are uploaded with the deployment from the local folder (or can move to blob storage).
- Database changes: generate a migration with drizzle-kit, apply it to Neon with `drizzle-kit migrate`; `db:reset` reseeds.

16.3 Data pipelines (run occasionally)

`npm run images:fetch`	Downloads openly licensed Commons photos (CC BY, CC BY-SA, CC0, PD) via the official APIs, keeps author and licence, with a hand-reviewed reject list.
`npm run tiles:build`	Extracts overview and core PMTiles for all cities (needs the pmtiles CLI).
`npm run map:assets`	Downloads label fonts and POI sprites (Protomaps assets).
`npm run routes:cache`	Pre-computes real routes from OSRM (one request per second, resumable).

17. Known limits and honest boundaries

- All catalogue content (hotels, flights, prices, transfers) is **sample data**; landmarks, coordinates and photos are real.

- Payments are test payments. No booking, change or cancellation happens on any real platform; the traveller does that and records it.
- Offline limits: no live traffic or transit, no address search, no re-routing on the phone, no background navigation in a PWA; iOS may evict stored data (the app repairs it when online); SMS needs cellular signal.
- Uploaded-plan quality depends on the documents: scanned PDFs without text need a photo; anything not printed in the documents is left blank and flagged; guides for uploaded plans currently use the standard template when the AI draft does not pass validation.
- Rate limiting is per serverless instance; the agent-desk login is a demo account; password reset and email verification need an email sender key; the in-memory SMS poll throttle is per instance.
- Third-party services used politely and for build time or low volume: Photon geocoder, OSRM routing, Protomaps build, Wikimedia Commons. For heavy production use they should be replaced with contracted or self-hosted equivalents.

18. End-to-end user journeys

A. Browse, select and travel (catalogue)

1. Visitor lands on the story, signs in (demo or email + password).
2. Searches the catalogue, opens an itinerary (photos, day plan, real map preview, price, offline size).
3. Taps Select: server creates a draft booking and builds the pack; the phone downloads and verifies it; "Offline guide ready".
4. Pays (test payment) when ready; the booking is confirmed.
5. Travels: day plan, map with GPS, guides and SOS work offline; the offline assistant answers from the saved bundle.

B. Add an existing travel plan

1. Taps Add travel plan, chooses several files of any supported format.
2. Watches the steps: reading with Gemini, finding places, making the offline street map, creating the trip, building the guide.
3. Reviews the "Please check" warnings and opens the trip; the bundle downloads to the phone; everything works offline.

C. Change a flight or hotel

1. Chooses "I'm logged in to Goibibo" in the chat, asks "move my flight later".
2. Agent shows ranked sample fares with each platform's price; taps "Pay on Goibibo", pays there.
3. Records the booking, reviews the card, taps Yes; the trip, totals and offline guide update.

D. Emergency

1. No data: the traveller opens the SOS screen (address card, emergency numbers) and taps SOS; or sends a text to the helpline.
2. The case reaches the Neon database when connectivity returns (app) or when the server reads the text (SMS).
3. The agent desk raises a red alert with the traveller, trip and location; a human takes over and resolves it.

Document generated from the project's source and README. Figures such as test counts, file sizes and timings are from the latest builds and live tests and will change as the project evolves.